

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

PROJECT REPORT

An Uncertainty-Aware Probabilistic Load Balancing Approach Using Server Metrics

Master of Technology

Department of Computer Science and Engineering

Submitted by

Parthajeet Deva Sarmah
Roll No.: 26CSM1R15

Abhay Nagesh Rao
Roll No.: 26CSM1R01

National Institute of Technology Warangal

October 16, 2026

1 Abstract

Cloud applications often run on multiple servers with different resource capacities and changing workloads. Conventional load-balancing methods, such as Round Robin and Least Loaded scheduling, make decisions using fixed allocation patterns or current resource measurements. These approaches may not adequately account for fluctuations in server load: two servers with similar average utilization can have different levels of variability and, consequently, different risks of overload.

This report proposes a lightweight, uncertainty-aware probabilistic load-balancing method that uses recent CPU utilization, memory utilization, and response-time measurements. A rolling observation window is used to estimate the mean and variability of each server metric. Under a simplifying normal-distribution assumption, the method estimates threshold-exceedance risk from the recent distributions of CPU and memory utilization. These estimates are combined into an overload-risk score, which is incorporated into a risk-adjusted server-selection score. They are heuristic estimates, not calibrated forecasts of post-assignment behavior. A simple admission rule discourages assignments to servers with high estimated risk when safer alternatives are available.

The method will be evaluated in a small simulation against several lightweight baselines, including Round Robin, a risk-free weighted-metric policy, Least CPU Utilization, Weighted Round Robin, Random Selection, and Power of Two Choices, under stable and bursty workloads. The evaluation will consider response time, throughput, service-level agreement (SLA) violation rate, CPU utilization, and load imbalance. The intended contribution is a practical demonstration of whether resource variability and estimated overload probability can improve server-selection decisions without requiring a complex machine-learning model or a real cloud deployment.

2 Problem Statement

A load balancer distributes incoming tasks among a set of available servers. The quality of its decisions affects response time, throughput, resource utilization, and the likelihood of overload or SLA violations.

A basic Least Loaded policy may choose the server with the lowest current utilization. However, an instantaneous measurement does not indicate how stable that server's recent workload has been. The proposed method therefore estimates threshold-exceedance risk from a fitted distribution of recent observations. This is a risk proxy, not a direct forecast of what will happen after a particular task is assigned. For example, two servers may both report 60% CPU utilization, while one has remained near 60% and the other has fluctuated substantially. A policy based only on their current values may treat them as equivalent even though their estimated overload risks differ.

The problem addressed in this project is therefore:

> **How can recent server metrics and their variability be used to estimate overload risk and improve task assignment compared with simple load-balancing policies?**

The project is scoped as a simulation-based implementation and comparison, rather than a production cloud scheduler or a claim of a new general-purpose probabilistic algorithm. It will investigate whether including estimated overload probability in server selection is beneficial, particularly when demand is bursty.

3 Proposed Solution

3.1 System Overview

The simulated system consists of a task stream, a collection of servers with potentially different capacities, a metric collector, and a probabilistic load-balancing component. At each scheduling decision, the component uses recent observations to score eligible servers and assigns the next task to the server with the lowest score.

The proposed workflow is:

1. Generate or receive an incoming task.
2. Collect each server's recent CPU utilization, memory utilization, and response-time observations.
3. Estimate the mean and standard deviation of each metric over a rolling window.
4. Estimate CPU and memory threshold-exceedance probabilities.
5. Combine the probabilities into an overload-risk score.
6. Calculate a risk-adjusted server score and select an eligible server.
7. Simulate task execution, update server metrics, and record performance.

3.2 Server Metrics and Rolling Statistics

For server i, define its observed metric vector at time t as

$$x_i(t) = [C_i(t), M_i(t), R_i(t)],$$

where $C_i(t)$ is CPU utilization, $M_i(t)$ is memory utilization, and $R_i(t)$ is response time.

A rolling window of the latest W observations is maintained for each metric. In the initial implementation, $W = 10$ is a configurable starting value rather than an empirically established optimum.

For CPU utilization, the rolling mean is

$$\mu_{C_i} = \frac{1}{W} \sum_{t=1}^{W} C_i(t),$$

and the rolling standard deviation is

$$\sigma_{C_i} = \sqrt{\frac{1}{W} \sum_{t=1}^{W} (C_i(t) - \mu_{C_i})^2}.$$

Equivalent statistics are calculated for memory utilization. The mean describes recent typical utilization, while the standard deviation describes observed variability.

3.3 Estimating Overload Probabilities

For the first version, CPU and memory observations are approximated using normal distributions fitted from their rolling means and standard deviations. Let the CPU threshold be $\theta_C = 80\%$ and the memory threshold be $\theta_M = 85\%$. These are configurable simulation parameters, not universal operating limits. Utilization is bounded between 0% and 100%, and a window of 10 observations is small; the normal approximation may therefore be inaccurate, particularly during abrupt workload changes. Treat the outputs as heuristic risk estimates, not calibrated real-world probabilities.

The estimated CPU threshold-exceedance probability is

$$P_{C_i} = 1 - \Phi\left(\frac{\theta_C - \mu_{C_i}}{\sigma_{C_i}}\right),$$

where Φ is the cumulative distribution function of the standard normal distribution. Similarly,

$$P_{M_i} = 1 - \Phi\left(\frac{\theta_M - \mu_{M_i}}{\sigma_{M_i}}\right).$$

If a metric has zero estimated variance, the implementation will use a deterministic threshold check to avoid division by zero.

The two probabilities are combined into an approximate overload-risk score:

$$P_i^{\text{risk}} = 1 - (1 - P_{C_i})(1 - P_{M_i}).$$

This combination assumes that CPU and memory threshold-exceedance events are independent. Real resource metrics can be correlated, so this assumption is a limitation of

the initial model. The score is an estimate based on recent observations, not a guarantee about future behavior.

3.4 Risk-Adjusted Server Selection

Let $\bar{R}_i$ denote the mean response time of tasks completed on server i during the rolling window, and let C_i and M_i denote the latest observed CPU and memory utilization percentages. Normalize response time using the positive reference value R_{ref}:

$$r_i = \frac{\bar{R}_i}{\bar{R}_i + R_{\text{ref}}}.$$

A current-load score is defined as

$$L_i = 0.4\frac{C_i}{100} + 0.4\frac{M_i}{100} + 0.2r_i.$$

The proposed risk-adjusted score is

$$S_i = 0.6L_i + 0.4P_i^{\text{risk}}.$$

The scheduler selects the eligible server with the lowest S_i. The coefficients are initial design choices intended to make the method simple and interpretable; they will be documented and, if time permits, varied in a small sensitivity analysis. They are not claimed to be optimal.

To avoid routinely assigning tasks to servers with high estimated combined overload risk, the scheduler will first exclude servers with $P_i^{\text{risk}} > 0.5$ when at least one lower-risk server is available. If all servers exceed the threshold, it will select the server with the lowest risk-adjusted score and record that the assignment was made under capacity pressure. The threshold is configurable and will be applied consistently during evaluation.

3.5 Baseline Policies

The proposed method will be compared with the following lightweight baselines:

- **Round Robin:** assigns tasks to servers in a repeating order.
- **Risk-free weighted-metric baseline (Least-Loaded variant):** selects the server with the lowest L_i score and does not use the probabilistic risk term. This is a project-defined baseline, not a claim to represent every conventional Least Loaded algorithm.
- **Least CPU Utilization:** selects the server with the lowest current CPU utilization. This tests whether CPU alone is a sufficient scheduling signal.

- **Least Memory Utilization:** selects the server with the lowest current memory utilization. This tests whether memory pressure alone is a useful scheduling signal.
- **Weighted Round Robin:** assigns tasks in proportion to configured server-capacity weights. The weights must be fixed before the experiment.
- **Random Selection:** chooses an eligible server uniformly at random, providing a simple reference point.
- **Power of Two Choices (P2C):** samples two eligible servers at random and assigns the task to the one with the lower current-load score L_i. This tests whether limited load awareness improves on random selection.

The risk-free weighted-metric baseline is especially important because it is matched to the proposed method's non-probabilistic score, helping isolate the effect of adding the risk term. For a short project, implement Round Robin, the risk-free weighted-metric baseline, Least CPU, Weighted Round Robin, Random Selection, and P2C first. Add Least Memory and Least Active Tasks only if time permits. All policies must receive the same task sequence, server capacities, and workload realization in each matched run.

3.6 Simulation and Experimental Procedure

The implementation will use a lightweight Python simulator with NumPy and pandas for calculations and Matplotlib for plots. A starting configuration will use five heterogeneous simulated servers and synthetic tasks with different resource demands. Two workload scenarios will be considered:

1. **Stable workload:** task arrivals and resource demands change relatively gradually.
2. **Bursty workload:** task arrivals or demands temporarily increase, creating periods of elevated resource pressure.

The task stream, server configuration, and random seed will be held constant across policies within each experimental run. Multiple random seeds will be used where time permits. The simulation will record task completion times, throughput, SLA violations, CPU utilization, and server-level utilization at each interval.

The normal approximation, thresholds, rolling-window length, and score weights will be stated explicitly. The simulator must define task-arrival generation, task CPU/memory demands and service durations, queue handling, and how utilization and response time are updated. These rules must be identical across policies. At the end of a run, either drain the queue until all accepted tasks finish or count unfinished tasks explicitly; otherwise response-time and SLA comparisons may be biased by tasks still waiting when the simulation stops. Results will be reported as measured outcomes; no improvement will be assumed in advance.

4 Dataset

No external dataset is required for the initial project. The evaluation will use **synthetically generated workload and server-metric data** produced by the simulator.

The synthetic environment will contain five servers with different processing capacities. Tasks will be assigned resource-demand parameters, such as CPU demand, memory demand, and expected processing duration. The simulator will update server utilization and response time as tasks arrive and complete. Stable and bursty workload patterns will be generated to test the policies under different levels of demand variation.

A fixed random seed will be used to make each workload reproducible. For fair comparison, every baseline and the proposed method will be tested on the same workload realization. The generated data will include timestamps, task identifiers, server assignments, CPU utilization, memory utilization, response time, completion status, and SLA status.

This synthetic dataset is appropriate for a short, controlled implementation project, but it does not establish that the method will behave identically on production cloud infrastructure. Using real monitoring traces could be considered a future extension if suitable data becomes available.

5 Evaluation Metrics

The proposed method will be evaluated against the baselines described above using the following metrics.

5.1 Average Response Time

The mean time between task submission and task completion is

$$\overline{R} = \frac{1}{K} \sum_{j=1}^{K} R_j,$$

where K is the number of completed tasks and R_j is the response time of task j. Lower values indicate faster completion on average.

5.2 Throughput

Throughput is the number of tasks completed per unit of simulated time:

$$T = \frac{\text{Number of completed tasks}}{\text{Simulation duration}}.$$

Higher throughput indicates that the system completes more work during the evaluation interval.

5.3 SLA Violation Rate

The fraction of completed tasks whose response time exceeds a configured SLA target R_{SLA} is

$$V_{\text{SLA}} = \frac{\#\{j : R_j > R_{\text{SLA}}\}}{K}.$$

A lower violation rate indicates that fewer completed tasks miss the response-time target. The SLA target will be fixed before comparing policies.

5.4 Average CPU Utilization

The time-averaged mean CPU utilization across all servers is

$$\overline{C} = \frac{1}{N\,T_{\text{sim}}} \sum_{i=1}^{N} \int_0^{T_{\text{sim}}} C_i(t)\,dt,$$

where N is the number of servers and T_{sim} is the simulation duration. Utilization will be interpreted alongside throughput and SLA violations; higher utilization alone does not necessarily indicate better performance.

5.5 Load Imbalance

The dispersion of server CPU utilization at time t can be measured using the population standard deviation:

$$I(t) = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (C_i(t) - \bar{C}(t))^2},$$

where $\bar{C}(t)$ is the mean CPU utilization across servers at time t. The reported imbalance will be the average of $I(t)$ across the simulation. Lower values indicate a more even distribution of percentage CPU utilization, not necessarily an even distribution of absolute work. Because servers are heterogeneous, interpret this metric alongside capacity, response time, throughput, and overload frequency; lower imbalance is not automatically better.

5.6 Estimated Overload Risk and Observed Overload Frequency

For diagnostic purposes, the experiment will record the estimated overload probability used by the scheduler and the observed fraction of measurement intervals in which CPU or memory exceeds its configured threshold. Comparing these values can help assess whether

the risk estimates are informative, although a small synthetic experiment cannot establish general calibration.

5.7 Evaluation Procedure

Each policy will be evaluated under both stable and bursty workloads using the same server configuration and task sequence for each matched run. Results will be summarized in tables and plots. The primary outcome of interest is whether the proposed method reduces SLA violations and observed overload frequency under bursty demand without causing unacceptable reductions in throughput or increases in response time.